

RAG Pipeline: PDF Chunking and Retrieval

Application Design Analysis Report

1. Application Overview

This application implements a Retrieval-Augmented Generation (RAG) pipeline that allows users to query a set of PDF documents using natural language. Rather than relying solely on a large language model's (LLM) pre-trained knowledge, the system grounds every response in content retrieved directly from the supplied documents, reducing hallucination and enabling domain-specific question answering without fine-tuning.

The pipeline is composed of five stages: (1) PDF loading via LangChain's PyPDFLoader, (2) recursive character-based text chunking, (3) vector embedding using OpenAI's text-embedding-3-small model, (4) local FAISS vector storage and similarity search, and (5) answer generation via gpt-4o-mini conditioned on the retrieved context.

2. Architecture Summary

Component	Technology	Role
Document Loader	LangChain PyPDFLoader	Parses PDF files into page-level Document objects
Text Splitter	RecursiveCharacterTextSplitter	Splits pages into 500-char chunks with 50-char overlap
Embedding Model	OpenAI text-embedding-3-small	Converts text to 1,536-dim float vectors
Vector Store	FAISS (local, CPU)	Indexes vectors and performs cosine similarity search
LLM	OpenAI gpt-4o-mini	Generates answers from retrieved context
Orchestration	LangChain Core	Chains retriever, prompt, LLM, and output parser

3. OpenAI API Calls

The pipeline makes three distinct calls to the OpenAI API. The first call (index build) embeds all document chunks in batch and occurs only once — results are persisted to disk. The second and third calls occur on every user query: the question is embedded for similarity search, and the retrieved context plus question are sent to the chat completions endpoint for answer generation. The FAISS similarity search itself runs entirely locally with no API involvement.

4. Design Strengths

- **Grounded responses:** Answers are derived from retrieved document content, significantly reducing LLM hallucination compared to prompt-only approaches.
- **Persistent index:** The FAISS index is saved to disk after the first build. Subsequent runs load it instantly, eliminating redundant embedding API calls and cost.

- **Local vector storage:** FAISS runs on-device with no external database dependency, keeping retrieval fast, private, and free beyond the initial embedding cost.
- **Source transparency:** Retrieved chunks are displayed with their source filename and page number alongside a similarity score, making answers auditable and traceable.
- **Portability:** The pipeline runs identically on a local machine and Google Colab with no infrastructure changes — a single install script handles dependencies.
- **Modular design:** Each stage (load, chunk, embed, index, query) is an independent function, making it straightforward to swap components — e.g., a different embedding model or a different vector store.

5. Design Limitations

- **Fixed chunk size:** The 500-character chunk size is a static heuristic. Chunks that split mid-sentence or mid-table can degrade retrieval quality. Semantic or sentence-aware chunking would improve coherence.
- **No re-ranking:** Retrieval relies solely on vector similarity (L2 distance). A cross-encoder re-ranker applied after initial retrieval would improve precision, especially for ambiguous queries.
- **Top-k only:** The pipeline always retrieves exactly k=3 chunks regardless of their relevance scores. A score threshold would prevent low-quality chunks from being included in the context.
- **Single-turn queries:** There is no conversation memory. Each question is answered independently, so follow-up questions that reference prior answers are not supported.
- **PDF text extraction limits:** PyPDFLoader extracts plain text only. Scanned PDFs, tables, figures, and complex layouts may be extracted poorly or lost entirely.
- **OpenAI dependency:** Both embedding and generation require an active OpenAI API key with sufficient credits. The pipeline cannot run fully offline.

6. Conclusion

This RAG pipeline demonstrates a practical and cost-effective approach to document-grounded question answering. The combination of local FAISS storage, persistent indexing, and OpenAI's embedding and chat models provides a strong baseline suitable for academic and small-scale production use. Key areas for improvement include semantic chunking, retrieval re-ranking, score thresholding, and multi-turn conversation support — each of which would meaningfully increase answer quality and user experience at modest additional complexity.